Backtesting platform UI/UX design plan

Oct 3, 2026 · @qiwen

1. Goals and design principles

This plan defines the web interface for the backtesting system described in *Backtesting System Architecture* (Oct 2, 2026). The web UI is the researcher's primary, interactive entry point: strategies are composed, run, analyzed, compared and researched in the browser, while the Python SDK and CLI remain available for scripting. The backend is fixed; this plan covers the interface layer and the API surface it needs from Layer 7.

The goal is to make the research loop of the architecture (compose a config, run it, read the result, change one slot, compare) fast and honest: from "why did the book lose that month" to the specific trade and the component action behind it in no more than 3 clicks, and from a promising result to its robustness checks in one click.

Success metrics

Metric	Target
Steps from changing one slot's variant or parameter to a submitted run	⩽ 2 (edit, run)
Clicks from a metric card to the component action that opened or closed a trade	⩽ 3
Launching the standard robustness set for a run	1 click
Launching an ablation, variant swap or counterfactual exit for a slot	1 click from the run or trade
Preview of a config's behavior around a chosen date	⩽ 10 s
State preserved across tab switches (time range, filters, selection, collapsed panels)	100%
Any run, trade or comparison reachable by URL with view state restored	100%
New user composes and runs a config from registered variants without help	< 10 min

Seven design principles

1. One focal point per screen. Each page has a single visual hero; everything else steps down in size, weight and contrast.
2. Runs are immutable. Every change produces a new run identified by its `run_id`; results are never overwritten or deleted, only archived.
3. The backend computes every number. The UI renders results from Layer 6 and never derives metrics, P&L or attribution itself, so what the screen shows is what the system reproduces.
4. Realism assumptions are tested, never tuned. λ, decision latency, participation cap, mark method and cleaning rules are shown read-only on every result and varied only through robustness runs, as the architecture's research register requires.
5. Drill down without navigating away. Details open in a right-hand drawer or as highlights on the chart; filters and selection persist.
6. The research protocol is built into the interface. Hold-out data, trial counts, confidence intervals and minimum trade counts are enforced or displayed by default, not left to discipline.
7. Consistency comes from the system. Spacing, type, color and chart style are tokens wrapped in components.

2. Users and core scenarios

The researcher is the primary user and works in the browser all day. Reviewers and new team members reach the product mostly through shared links, the comparison view and the robustness panel.

User	Typical behavior	What they need from the interface
Researcher (primary)	Composes configs from registered variants, runs dozens of backtests and sweeps a day, researches one slot at a time, traces trades back to component actions	A fast compose, run, inspect, compare loop; one-click research actions; drill-down that never loses context; keyboard operation
Portfolio manager / reviewer	Reviews candidates a few times a week, challenges realism assumptions, checks the pre-launch checklist	One link to the conclusion; realism strip and protocol flags visible at a glance; robustness results side by side

User	Typical behavior	What they need from the interface
New team member	Learns the system, reproduces runs, reads how components behave	A traceable experiment log; preview and decision traces that explain behavior; empty states that guide

Frequent tasks and success criteria

Task	Trigger	Success criterion
Change one slot's variant or parameter and rerun	The most frequent action in research	2 steps; the new run differs from its baseline in that slot only and opens in comparison with one click
Find why the book lost money in a period	A dip on the equity curve	After brushing the period, the trades, their closing components and Greeks attribution are 3 clicks away
Test whether a result survives realism stress	A promising run	The standard robustness set launches in 1 click and reports pass or fail per item
Research one component	Deciding whether a slot adds value	Ablation, variant swap, slot sweep or counterfactual exits launch from the run or trade and land as paired runs
Compare 2 to 4 runs	Choosing between versions	Full config diff highlighted; equity, metrics and confidence intervals side by side
Check readiness for paper trading	Before promotion	The pre-launch checklist shows each item as pass, fail or not run, with a button for missing tests
Share a trade or a run	Discussing an anomaly	Copy a link; the recipient sees the same view state
Reproduce a run	Onboarding or audit	Config, commit, data snapshot and result hash shown; rerun from the log in one click

Design priority follows task frequency: the first two rows shape the composer and the analysis page; the research and robustness rows shape section 8.

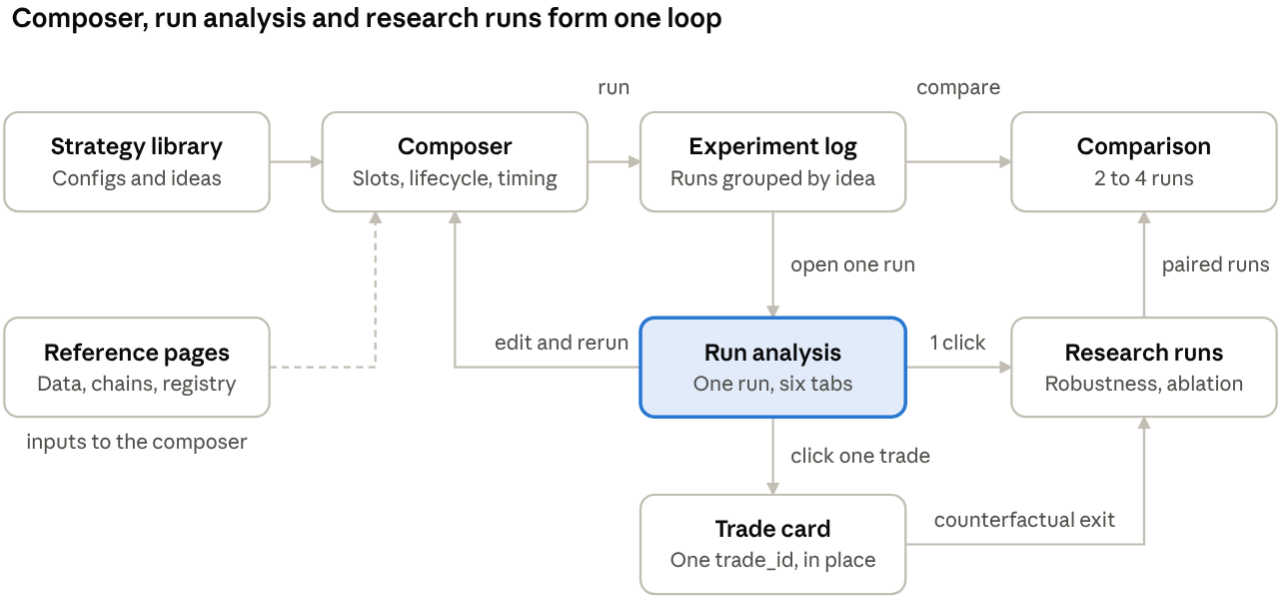
3. Information architecture

Pages are views of the backend's objects. The interface uses the architecture's vocabulary throughout, so a researcher never translates between what the screen says and what the logs, configs and API say.

Object model

Object	Backend source	Views
Idea	A named group of runs in the ExperimentTracker; carries the trial count used by the Deflated Sharpe	Strategy library, experiment log grouping
Config	Strategy (7 slots: variant, version, params), lifecycle (5 settings), timing (2), run settings, data snapshot ID	Composer, YAML view, config diff
Variant	A registered component with a name, version and parameter schema	Component registry, slot cards
Run	Immutable; `run_id` is the hash of config, code commit, data snapshot and run settings; mode is backtest, paper or live	Experiment log, run analysis, comparison
Sweep	A set of runs from `POST /sweeps` sharing a base config	Sweep view inside run analysis, sensitivity heatmaps
Trade	One `trade_id` in the TradeBook: setup context, entry, every action with its component, MFE and MAE, legs with main or hedge role, shadow tracking	Trade table, trade card
Candidate	Per-underlying setup state (Idle or Armed) before a trade opens	Preview, trade card lifecycle
Snapshot	Daily equity, cash, positions, premium at risk, net Greeks, turnover	Charts in run analysis
Data snapshot	A dataset version with its quality report	Data quality page, realism strip

Screen map



Screen map · 8 surfaces, one research loop

The composer, experiment log and run analysis form the main path; "edit and rerun" sends the user back to the composer. Research runs (robustness set, ablations, variant swaps, sweeps, counterfactual exits) launch from the analysis page or a trade card and land as paired runs that open in comparison. Reference pages (data quality, chain and surface explorer, component registry) feed the composer.

URL state

The URL encodes the object, current tab, time range, filters, selection and drawer state, so a copied link opens the same view.

- `/ideas/{id}`, `/compose/{config_id}`, `/runs/{run_id}?tab=overview&range=2024-07-15..2024-08-30&trade=T-0132`, `/compare?runs=a1f3,b7c2,c9d0`, `/sweeps/{id}`
- Tab, time brush, selected `trade_id` and drawer state are written to the URL; browser back and forward work
- Comparison links carry the baseline column and the visible sections

Global shell

Region	Spec	Content
Left navigation rail	56 px icon rail, expands to 240 px	Ideas, composer, experiment log, data, registry, settings, search

Region	Spec	Content
Top context bar	48 px, fixed	Breadcrumb (idea, run), mode badge, realism strip, protocol flags, primary action
Right drawer	400 px, draggable to 560 px, expandable to full screen	The selected trade, run or variant
Center area	Fluid, minimum width 1100 px	The current page
Job tray	Bottom right, collapsible	Queued and running jobs with progress and cancel

4. Page design: strategy composer

A strategy is a config, not code: each of the seven slots is filled by a registered variant with parameters, plus lifecycle, timing and run settings. The composer is a full interactive editor for that config. Variant code itself is written and unit-tested outside the UI and appears here through the registry. The composer's hero is the slot board; the goal is to make "change one slot and rerun" a 2-step action.

Layout

Three columns, default ratio 60 / 25 / 15 on wide screens; dividers are draggable and remember their position.

Region	Position	Content
Slot board	Left, main area	Seven slot cards arranged along the trade lifecycle: Idle (1 Setup / Regime), Armed (2 Entry, 6 Positioning), Open (3 Take Profit, 4 Stop Loss, 5 Headwind, 7 Hedge / Protection)
Settings panel	Right	Collapsible groups: lifecycle, timing, run settings, data snapshot, book limits
Preview and validation	Bottom, draggable height	Preview tab, validation tab, YAML tab

Slot cards

- Each card shows the slot name, the selected variant and version, a one-line summary of its parameters, and whether it differs from the baseline run
- Clicking a card opens it in place: a variant picker (name, version, description, last-used date, unit-test status from the registry) and a parameter form generated from the variant's `ParamSchema`, with types, defaults and valid ranges
- A neutral variant is always available, so any slot can be switched off for an ablation from the card itself
- Book-level slots (5 Headwind and 7 Hedge on the whole book, 6 Positioning) show a small "book" tag, since they run in Layer 3
- The Open-state cards display the current `open_state_order` as a numbered sequence; dragging cards reorders it and updates the lifecycle setting
- Slot interactions declared by the variants (pass-through, request, override channels) are drawn as thin links between cards on hover, so the researcher sees which slots depend on each other before changing one

Settings panel

Group	Fields	Notes
Lifecycle	`open_state_order`, `conflict_policy`, `max_concurrent_trades`, `reentry_policy`, `check_frequency`	`check_frequency` carries a fixed note: must equal live trading
Timing	`signal_snapshot`, `fill_latency`	Set once per config; a warning appears if they differ from the team's live settings
Run settings	Period, initial capital, cost model	Period selection shows the locked hold-out as a shaded, unselectable range
Data snapshot	Snapshot ID picker	Shows the quality summary of each snapshot; defaults to the latest validated one
Book limits	Total premium, premium per underlying and expiry, Greeks limits, scenario loss limit	Values from Layer 3 constraints and scenario rules

Every field changed from the baseline shows a small dot; hover to see the baseline value and reset. A line at the top reads "Differs from baseline in 2 slots, 1 lifecycle setting"; clicking expands the diff. Validation runs against the schema as the user types; unknown or out-of-range values are flagged in place, and the run button shows the blocking reason instead of being disabled.

YAML view

The YAML tab shows the exact config that will be submitted, in the architecture's format. Editing is two-way: changes in YAML update the cards and vice versa, and schema errors are pinned to the line. The tab also offers "copy as CLI command" for running the same config from a terminal.

Preview

The preview answers "what would this config do around this date" before a full run. The user picks a date; the backend runs a short engine pass (default 60 trading days ending at that date, with warm-up) with the decision log on, and returns within 10 s.

- A candidate table: each underlying's state (Idle or Armed) on the chosen date, the setup score and regime label
- Entry results: for armed candidates that fired, the selected contracts with quotes, spread, IV and Greeks at entry
- A compact action log for the window, filterable by slot, so the researcher can see which components acted and why
- A link from any contract to the chain explorer at that date

Submitting

1. "Run" submits the config through `POST /backtests` and adds the job to the job tray with queue position and estimated time.
2. "Sweep" opens a sweep builder: pick one or two parameters on any slot or lifecycle setting, set ranges or a search method (grid, random, Bayesian), see the number of runs and its effect on the idea's trial count before submitting through `POST /sweeps`.
3. The composer stays open after submitting, so the next variation can be prepared while the run queues.
4. When the run finishes, the job tray offers "Open" and "Compare with baseline".

5. Page design: experiment log and comparison

The experiment log is the browser face of the ExperimentTracker. Runs are grouped by idea, because the idea's trial count is what the Deflated Sharpe needs and what keeps research honest.

Experiment log list

Ideas are collapsible groups, newest activity first; each group header shows the idea name, total trials, best Deflated Sharpe and hold-out status (locked or revealed on a date). Inside, one row per run; columns are configurable, sortable and freezable.

Column	Content	Interaction
Run	Short `run_id`, user label, mode badge (backtest, paper, live)	Click opens run analysis; label editable inline
Config diff	What differs from the idea's baseline: slots, variants, params, lifecycle, timing, data snapshot, commit; up to 3 items	Hover shows the full diff
Research tag	Plain run, robustness item, ablation, variant swap, sweep member, counterfactual	Click filters to the same kind
Core metrics	Annual return, Sortino, max drawdown, trade count, return on premium, with a sparkline	Values come from Layer 6
Interval	90% bootstrap interval for the headline metric	Grey when below the minimum trade count
Realism	Short chips: λ, latency, mark method	Click filters by the same assumptions
Status	Queued, running, done, failed, invariant failure, archived	Running rows show progress
Note	One-line note	Editable inline

A filter bar filters by idea, slot, variant, research tag, data snapshot, commit, metric ranges and status; filters are written to the URL. Ticking 2 to 4 rows raises a "Compare" button. Runs can be archived but never deleted: archived runs are hidden by default and still count toward the idea's trials.

Comparison view

Ordered "differences first, results second", in four sections.

1. Config diff: the full config tree (seven slots, lifecycle, timing, run settings, data snapshot, commit), with differing values highlighted and identical branches collapsed. Two runs that differ only in data snapshot or commit are flagged at the top, since that is a reproducibility event, not a research change.
2. Metrics: rows are metrics, columns are runs; each cell shows the value and its bootstrap interval; overlapping intervals are marked so a difference that is not significant does not look like a win.
3. Equity and drawdown overlay: one line per run, colors in column order; the hold-out range is shaded and blank unless revealed.
4. Trade-level diff, for runs that differ in one slot: trades present in only one run, trades whose exit changed, and the P&L effect of each, which is the direct readout of what that slot changed.

The view supports adding a run, reordering columns and setting a baseline column. State is written to the URL.

Labels and promotion

Runs carry labels such as "candidate" or "rejected". A run can be promoted to paper trading only from the pre-launch checklist (section 8), so promotion always passes through the same gate.

6. Page design: run analysis

The analysis page is the most used and densest page; its hero is the book's equity curve built from daily mark-to-market, which is how the architecture computes every risk and return metric. Three tiers top to bottom: conclusion, process, detail, all charts on one time axis.

Three-tier layout

Tier	Height	Content
Top: conclusion	Fixed, about 128 px	Realism strip, protocol flags, metric cards (max 8)
Middle: process	Fluid; each layer collapsible and resizable	Chart stack on a shared time axis

Tier	Height	Content
Bottom: detail	280 px default, draggable	Trade table or trade lifecycle Gantt; a row opens the trade card

The top tier is sticky; on scroll the metric cards shrink to one line.

Realism strip

One read-only line restates what determined this result: data snapshot ID, code commit, period, capital, signal snapshot time, fill latency, λ, participation cap, mark method, `check_frequency`, and the result hash. Each chip shows its value and, on hover, which robustness runs exist for it and whether they passed. Chips are not switches: changing a realism assumption is done by launching robustness runs (section 8), never in place.

The one display toggle on the page is mark-to-market at bid or mid. Both series are stored by Layer 5, so switching changes only the drawn equity and drawdown; a label makes clear that metrics and stop triggers were computed on the run's configured mark method.

Protocol flags

A small row of flags after the realism strip: trade count below the minimum, hold-out not yet tested, profit concentrated in the top N trades, Greeks attribution not matching the thesis, invariant warnings, data-quality issues in the snapshot. Each flag expands to the explanation and a link to the relevant chart or test.

Metric cards

- Default set: annualized return, Sortino, max drawdown, return on premium, win rate, payoff ratio, share of profit from the top 5 trades, Deflated Sharpe
- Each card shows the value, a 90% bootstrap interval as a thin bar beneath it, and the delta versus the run's baseline
- Cards turn muted when the trade count is below the minimum, with the reason on hover
- Hover shows the definition and the trial count used for the Deflated Sharpe; card selection is configurable

Tabs

Tab	Main chart	Supporting views
Overview	Equity vs benchmark (log toggle), hold-out shaded	Underwater drawdown, monthly returns heatmap, rolling Sortino

Tab	Main chart	Supporting views
Exposure	Premium at risk vs budget	Net Greeks by underlying and expiry bucket, beta-weighted delta and vega vs limits, risk events on the timeline
Attribution	Greeks P&L attribution (delta, gamma, vega, theta, residual)	P&L by closing component, by main vs hedge legs, by underlying; residual outliers listed for investigation
Trades	Trade lifecycle Gantt with action markers	Trade table, MFE and MAE scatter, IV at entry vs realized volatility over the hold
Execution	Spread paid as % of premium over time	Rejected and partial fill share, participation distribution, costs as share of gross P&L
Robustness	Robustness set results (section 8)	Sweeps, baselines, validation folds, pre-launch checklist

Chart stack rules

- All layers share the x axis, time brush and crosshair; y axes are independent, labels inside the plot
- Each layer collapses to a 28 px title bar
- The tooltip is fixed at the top right and lists every layer's value at the hovered time
- Brushing filters every chart and the trade table; a chip shows the selected range with a clear button
- Timeline markers: earnings and events, expiries, risk events (scale down, no new positions, liquidate, pause), constraint vetoes; hover for details
- The hold-out range is shaded and empty until revealed
- Each layer exports PNG, CSV, or a link to the current range

Trade table

Default columns: `trade_id`, underlying, legs summary, opened, closed, holding days, P&L (main), P&L (hedge), closing component and action, MFE, MAE. Sorting, column filters and configuration; selected rows shade their holding period on the charts; arrow keys move, Enter opens the trade card.

7. Page design: trade card

The trade card is the browser view of one TradeBook record. It opens in the right drawer without leaving the analysis page. Its hero is the trade's P&L; the rest answers, in order, what the trade was, what each component did to it and why, and what would have happened otherwise.

Content structure

Block	Content
Header (sticky)	`trade_id`, underlying, opened and closed dates, holding days, previous and next, close
Conclusion	Total P&L split into main and hedge legs, closing component and action, return on premium
Lifecycle timeline	Idle, Armed, Open, Closed as a horizontal strip with dates; every action (OPEN, ADD, REDUCE, ROLL, HEDGE_ADD, HEDGE_REMOVE, SET_STOP, CLOSE) as a marker colored by the issuing slot; HOLDs collapsed into a count per slot
Price and P&L chart	Underlying path and the trade's P&L over the hold, with entry, actions and exit marked; stop levels from SET_STOP drawn as steps; the shadow-tracking period after exit shown dashed
Excursions	MFE and MAE in premium and in underlying terms, with the date each occurred
Context	The trade context by namespace (setup, entry, take_profit, stop_loss, headwind, positioning, hedge): each slot's written values at entry and at each action, timestamped
Action log	One row per action: time, slot, variant and version, action, fraction, reason, and the data values that triggered it; risk outcomes such as "vetoed by premium limit" shown inline
Legs	One row per leg: contract, side, quantity, role (main or hedge), entry and exit quotes, fill price with spread paid, fees, leg P&L
Attribution	This trade's Greeks attribution waterfall, residual highlighted when large

Block	Content
Counterfactual exits	Launch an alternative exit for this trade (another Take Profit or Stop Loss variant, hold to expiry, exit at a date); results labeled in-sample
Trace	The fill to order to target to signal to component chain from the decision log, collapsed by default
Note	A one-line note shown in the trade table

Interaction

- Opening the card scrolls the chart stack to the holding period and shades it without changing the brush
- J and K walk through trades in the table's current order
- Clicking a marker on the lifecycle timeline highlights the matching row in the action log and the context values written at that moment
- Clicking a slot name anywhere in the card filters the action log to that slot and offers "Ablate this slot" and "Swap variant" for the whole run
- The drawer is 400 px, draggable to 560 px, and expands to a full-screen two-column view
- The header menu offers copy link, locate on chart, open in chain explorer at entry date, add to watchlist

Batch view

A multi-row selection opens an aggregate card: count, total P&L, main vs hedge split, closing-component distribution, MFE and MAE scatter of the selection, and a per-trade P&L bar chart that opens single cards.

8. Research and robustness

The architecture builds research tools once in Layer 6 and reuses them for every component. The interface exposes each of them as a one-click action that produces runs differing from a baseline in one thing only, recorded in the ExperimentTracker and opened in comparison.

Robustness panel

The Robustness tab of a run shows the standard set from the architecture's research register. One button launches every missing item as a sweep; each item reports pass, fail or not run.

Item	Runs launched	Pass condition shown
Fill price multiple	λ = 0.5, 1, 1.5	Still positive at λ = 1.5
Decision latency	1, 5, 15 minutes	Decay curve; no collapse at the configured latency
Doubled costs	Cost model × 2	Still positive
Signal delay	One-bar delay	Still works
Participation cap	Several caps	Return and capacity curve
Mark method	Bid vs mid	Stop triggers and drawdowns compared
Data cleaning	With and without wide-spread day removal	Stress periods compared
Baselines	Delta-equivalent underlying; random entries with the same structure, delta and days to expiry	Beats the random-entry band
Validation	Walk-forward folds, purged k-fold, trade bootstrap	Out-of-sample does not collapse; intervals reported
IV regime split	Results by IV regime	No regime carries the whole result

Thresholds are team settings, not per-run choices, so a pass means the same thing for every run.

Research actions

Available from slot cards, the run header and the trade card; each opens a small dialog showing exactly which runs will be created and how many trials they add to the idea.

Action	What it creates	Where the answer appears
Ablate slot	The run with one slot filled by its neutral variant	Comparison with the trade-level diff
Swap variant	The run with one slot filled by another registered variant	Comparison
Slot sweep	A sweep over one slot's parameters, other slots fixed	Sensitivity heatmap: plateau vs isolated spike
Interaction test	A joint sweep over two slots (e.g. Take Profit × Stop Loss)	Heatmap of the joint effect next to the sum of single-slot effects; a large gap is flagged
Lifecycle comparison	Runs with different `open_state_order` or `conflict_policy`, slots fixed	Comparison
Counterfactual exits	Every closed trade replayed under an alternative exit, from shadow-tracking data	Money saved or given up per trade and in total; labeled in-sample
Event study	Setup and Entry judged by forward returns with no exits applied	Forward-return curves by horizon with intervals
Attribution by component	P&L split by the slot that opened, sized, reduced, hedged or closed each part of a trade	Waterfall by slot

Protocol guardrails

The research protocol in section 6.4 of the architecture is enforced or displayed by default:

- Hold-out: the period is locked per idea and shaded on every chart. Revealing it is an explicit action with a confirmation, logged with user and date, allowed once per idea; afterwards the idea header shows "hold-out revealed"
- Trial counter: every run, sweep member and research action increments the idea's count, archived runs included; the Deflated Sharpe always uses it, and research dialogs show the count before and after
- Intervals over points: metric cards, comparison cells and research results show bootstrap intervals; a difference whose intervals overlap is not highlighted as an improvement

- Minimum trade count: below the team threshold, metrics are muted and conclusions are labeled insufficient
- Per-underlying check: pooled results link to a per-underlying breakdown, and an effect that fails on any underlying is flagged
- In-sample labels: counterfactual results carry an in-sample badge and a button to confirm the change out of sample
- Staged order: the idea header shows the recommended research stage (Setup and Entry, exits, Headwind, Positioning, Hedge, interactions) and which stages have runs

Pre-launch checklist

A panel on the Robustness tab with the architecture's eight items, each marked pass, fail or not run, with a button that launches the missing test: out-of-sample holds; stable around chosen parameters; works with one-bar delay; positive with doubled costs and λ = 1.5; profit not concentrated; beats the random-entry baseline; Greeks attribution matches the thesis; total trials recorded. "Promote to paper" is enabled only when all eight pass, and promotion creates a paper-mode run that appears in the same idea.

9. Data and reference pages

Three pages support composing and diagnosing. They are reached from the left rail and from links in the composer, preview and trade card, always opening at the date and underlying of the context they came from.

Data quality

One page per data snapshot, built from the Layer 1 quality reports.

- Coverage calendar: a heatmap by date and underlying showing missing days, stale quotes, crossed or locked markets, very wide spreads and prices below intrinsic value
- Contract coverage: listed contracts per day, expired and delisted contracts retained, OCC-adjusted contracts excluded with their share and timing
- Cleaning log: every rule applied, with its version and the count of affected records
- Snapshot comparison: what changed between two snapshots, so a run that differs only in data can be explained

Chain and surface explorer

Point-in-time views from `GET /chains/{underlying}?date=` and the surface endpoint.

- Chain table: strikes by expiry with bid, ask, spread %, volume, open interest, IV and Greeks; filters mirror the Entry slot's filters, so the researcher sees what Entry could select
- Surface: IV by delta (or moneyness) and tenor as a heatmap with the smoothed surface and flagged arbitrage violations; ATM IV, skew, term structure and IV rank over time beneath
- Quote quality markers on every cell (stale, wide, crossed), consistent with the data quality page
- A date scrubber steps through days; a trade card link opens the explorer at that trade's entry date with its contracts highlighted

Component registry

A browsable list from `GET /components`: every variant by slot, with name, versions, description, parameter schema, unit-test status, and the runs and ideas that used it. A version diff shows changed parameters and their defaults. Selecting a variant offers "Use in composer" and "Swap into this run".

10. Interaction rules

Six rules govern interaction across the product. Any new page or component is checked against them first.

Rule 1: runs are immutable

Submitting after any change creates a new run with a new `run_id`. There is no overwrite. Runs are cancelled or archived, never deleted, so the trial count stays true.

Rule 2: the backend computes, the interface displays

Change	How it happens	Response
Slot variant, params, lifecycle, timing, run settings, data snapshot	New run from the composer	Queued job, progress in the tray
Realism assumption (λ, latency, cap, cleaning)	Robustness runs from section 8	Paired runs, results in the Robustness tab and comparison
Research question about one slot	Research action from section 8	Paired runs or sweep, opened in comparison

Change	How it happens	Response
Display only: bid or mid mark, log scale, benchmark overlay, visible layers	Toggle on the page	Instant; metrics unchanged and labeled as such

The interface never derives metrics, P&L or attribution itself; every number on screen comes from a Layer 6 or Layer 5 endpoint for a specific `run_id`.

Rule 3: drill down without navigating away

The chain is metric card, chart layer, time range, trades in that range, trade card, action, trace. Each level opens as a highlight or in the drawer; breadcrumbs step back; filters and selection persist. Target: metric card to the component action behind a trade in 3 clicks.

Rule 4: global linking

- One time brush controls every chart and the trade table on the page
- The crosshair is synchronized; a fixed tooltip lists every layer's value at the hovered date
- Selecting a trade shades its holding period; selecting a range scrolls the table to the first trade in it
- Clicking a slot anywhere (action marker, attribution bar, closing-component cell) filters the page to that slot's actions
- Tab switches keep time range, filters, selection and collapsed state

Rule 5: every research question becomes a comparison

Research actions, robustness items and sweeps always land as runs that open in comparison against their baseline, with the trade-level diff when they differ in one slot. The comparison URL is the shareable answer.

Rule 6: editing gives immediate feedback

Schema validation as the user types; the slot board updates from YAML and back; preview returns within 10 s; the run button switches to a queued state on click; the job tray shows progress without leaving the page.

Selection and hover states

State	Treatment
Hover	Background steps up one surface level
Selected	Light accent fill and a 2 px accent bar on the left
Multi-select	Checkbox per row; a bulk-action bar shows the count
Focus	1 px accent outline during keyboard navigation only
Disabled	Not used; controls stay clickable and explain what is blocking

Keyboard and command palette

Cmd/Ctrl + K opens the command palette: jump to an idea, run, `trade_id`, variant or underlying, and run actions (submit, compare, ablate slot, open robustness). Tables support arrows, Enter, Esc and Space. The full list is in the appendix and in button tooltips.

Context menus

Charts, table rows, slot cards and metric cards have context menus of 4 to 6 actions tied to the object, for example: locate on chart, copy link, ablate this slot, swap variant, run counterfactual exit, open in chain explorer.

11. State design

Every page and component is designed in all of its states. The table defines the uniform treatment.

State	Treatment	Never
Loading	Skeletons shaped like the final content, shown only after 300 ms	Full-page spinner, blank page
Queued	Job tray row with queue position and estimated start; the run's page shows the config and realism strip with "queued"	Pretending the run has started

State	Treatment	Never
Running	Progress bar from the job queue with elapsed and estimated remaining time; cancel always available	A modal blocking the page
Loading results	Staged: metric cards first, then the equity and drawdown charts, then the other layers and the trade table	Waiting for every endpoint before showing anything
Invariant failure	The engine stopped and dumped state: a red banner names the failed invariant and date, links the state dump and decision log, and offers rerun after fix	Showing partial results as if valid
Interrupted	Resumable from the last state snapshot: "Resume" next to "Rerun"	Losing completed work
Stale composer	The composer differs from the last submitted config: a line says so with a submit button	Discarding edits
Empty	One sentence on what will appear and one button to the next step (e.g. "Compose your first config", "Run the robustness set")	The words "no data" alone
Error	What happened and what to do, pinned to the field, line or endpoint; the rest of the page stays	Modals, raw exception text

Notifications

Finished runs and sweeps raise a toast with "Open" and "Compare with baseline", gone after 5 s and kept in the notification center. Failures and invariant failures stay until dismissed.

Partial data

Where a chart lacks data, the span is hatched and the hover explains why (no quotes, flagged untradable, outside the snapshot); lines never interpolate across it. Empty table cells stay empty.

Sweeps

A sweep page shows members as a grid that fills in as runs finish; finished members open immediately. On completion the sensitivity heatmap appears with plateau or spike annotations.

Concurrent editing

Composer edits auto-save as a draft config per user. When two people edit the same config, the later saver sees a config diff and chooses which version to keep.

12. Visual system

Dense but orderly: alignment, whitespace and hierarchy create order; a small palette carries meaning. Every value is a design token.

Typography

Use	Typeface	Size	Weight
Interface text	Sans-serif (Inter or system font)	13 px base, 12 px secondary	400
Page and section titles	Same	16 px page, 14 px section	500
Numbers, ids and code	Monospace (JetBrains Mono or system mono); tabular figures in the UI font	Matches surrounding text; 20 px on metric cards	400, 500 on metric cards
YAML editor	Monospace	13 px	400
Chart labels	UI font	11 px minimum	400

Two weights only. Line height 1.5 for body, 28 to 32 px table rows.

Color

Interface colors and data colors are separate systems.

Category	Tokens	Use
Surfaces	surface-0, surface-1, surface-2	Page, panel, card; regions separated by surface difference, not borders
Text	text-primary, text-secondary, text-muted	Body, supporting, hints
Borders	border, border-strong	0.5 px dividers; strong for hover and focus
Accent	accent	Primary actions, selection, links
Semantic	success, warning, danger	Status, pass and fail, P&L sign
Strategy and benchmark	data-strategy, data-benchmark	Equity line and benchmark, everywhere
Slots	slot-1 to slot-7	One fixed color per strategy slot, used for action markers, attribution by component, slot filters and slot cards
Greeks	greek-delta, greek-gamma, greek-vega, greek-theta, greek-residual	Exposure and attribution charts, everywhere
Leg roles	leg-main, leg-hedge	Main and hedge legs in tables and P&L splits
Comparison series	data-1 to data-4	Run columns in comparison
Hold-out	holdout-fill	Hatched shading of the locked period

P&L defaults to green up and red down, switchable in settings; sign is always carried by a + or − as well. Dark mode is the default; large fills drop saturation by 15% in dark mode. Text contrast is at least 4.5 : 1. Slot and Greek palettes are checked for color-blind separation, and every slot marker also carries its number.

Spacing and density

- 8 px grid: 8, 12, 16 inside components, 24 between blocks, 32 between page sections
- Compact (default) and comfortable densities
- Corners: 6 px controls, 8 px cards, 12 px drawers and popovers; no shadows
- Outline icons, 16 px inline, 20 px in navigation

Chart style

Element	Rule
Axes	x baseline only; light horizontal gridlines
Lines	Strategy 1.5 px, others 1 px; no smoothing
Areas	15% opacity of the line color
Intervals	Thin bars or bands at 20% opacity behind the point estimate
Heatmaps	Single-hue for one-directional metrics; diverging around 0 for signed ones; plateau regions outlined
Markers	Action markers in slot colors; events and risk events as 4 px triangles above the x axis
Tooltip	Fixed at the top right, monospace right-aligned
Missing data and hold-out	Gaps with hatching; never interpolated

Chart type mapping

Data	Chart
Equity	Line, log toggle, benchmark overlay
Drawdown	Underwater area
Premium at risk	Area with budget line
Net Greeks, beta-weighted exposure	Multi-line with limit lines
Greeks attribution, attribution by component	Waterfall or stacked bars over time
Monthly returns, sweeps, interactions, coverage	Heatmap
Trade lifecycle	Gantt with action markers
MFE vs MAE, IV at entry vs realized volatility	Scatter
Robustness decay (latency, λ, cap)	Line with intervals
IV surface	Heatmap by delta and tenor

13. Component inventory

Three layers: base controls, data components, page-level compositions. Pages are assembled from the upper two only. Each component ships with all its states in Storybook, and design review uses Storybook.

Layer	Component	Spec notes	Priority
Base	Button, input, select, toggle, chip	Three button levels; one primary button per page	P0
Base	Breadcrumb, tabs, tooltip, context menu	Tab switches never reset state	P0
Base	Drawer, toast, confirm dialog	Confirm only for irreversible actions such as revealing the hold-out	P0
Base	Draggable divider, collapsible panel	Remember adjustments; collapsed headers show a summary	P0
Data	Metric card with interval	Value, interval bar, baseline delta, muted state below minimum trades	P0
Data	Data table	Virtual scrolling, sticky header, sorting, filters, column config, frozen columns, keyboard navigation	P0
Data	Time-axis chart group	Shared x axis, brush, synced crosshair, fixed tooltip, event and action markers, hold-out shading	P0
Data	Realism strip and protocol flags	Read-only chips with robustness status on hover; expandable flags	P0
Data	Filter bar	Active filters, clear one or all, written to URL	P0
Data	Waterfall	Greeks and component attribution; total bar; slot and Greek colors	P0
Data	Lifecycle Gantt and timeline	State spans and action markers per `trade_id`	P0

Layer	Component	Spec notes	Priority
Data	Heatmap	Single-hue and diverging; plateau outline; cell click opens the run	P1
Data	Config diff tree	Collapsible config tree with differing values highlighted	P0
Data	Sparkline	List rows; no axes	P1
Input	Slot card	Variant picker, schema form, baseline dot, neutral option	P0
Input	Schema form	Generated from `ParamSchema`: types, defaults, ranges, inline validation	P0
Input	YAML editor	Two-way sync with the slot board, schema errors pinned to lines	P0
Input	Sweep builder	Parameter ranges, method, run count and trial-count impact	P1
Page-level	Composer layout, analysis layout, comparison layout	Fixed skeleton with content slots	P0
Page-level	Trade card	Section 7 blocks, single and batch	P0
Page-level	Robustness panel and checklist	Section 8 items with pass, fail, not run and launch buttons	P0
Page-level	Research action dialog	Shows runs to be created and trial-count impact	P0
Page-level	Job tray	Queue, progress, cancel, resume	P0
Page-level	Chain table and surface view	Point-in-time, quality markers, date scrubber	P1
Page-level	Command palette	Objects and actions, keyboard first	P1

14. Backend contract

The interface reads and writes only through Layer 7. This section maps each surface to the REST draft in the architecture (section 7.1) and lists what needs to be added. Nothing here changes Layers 0 to 6; the additions expose data those layers already produce.

Existing endpoints and their use

Endpoint	Used by
`POST /backtests`	Composer submit; research actions; robustness items
`GET /backtests/{run_id}`	Job tray status and progress; analysis header; metric cards
`GET /backtests/{run_id}/equity`	Equity and drawdown layers
`GET /backtests/{run_id}/trades`	Fills in the trade card's legs block
`GET /backtests/{run_id}/greeks`	Exposure tab; Greeks attribution
`POST /sweeps`	Sweep builder; slot sweeps; interaction tests; robustness set
`GET /components`	Slot cards, schema forms, component registry
`GET /datasets`	Data snapshot picker; data quality page
`GET /chains/{underlying}?date=`	Chain explorer; preview links

Additions needed

Endpoint or field	Purpose	Backend source
`GET /backtests/{run_id}/tradebook` with `?trade_id=`	Trade table and trade card: context namespaces, actions with component and reason, MFE and MAE, legs with role, main and hedge P&L, shadow-tracking series	Layer 5 TradeBook
`GET /backtests/{run_id}/snapshots`	Premium at risk, cash, positions, turnover, bid and mid marks	Layer 5 Snapshots and Valuation
`GET /backtests/{run_id}/log?level=decision&trade_id=`	Trace and action log	Layer 0 decision log
`GET /backtests/{run_id}/metrics` with intervals and trial count	Metric cards, protocol flags, comparison	Layer 6 Metrics and Validation
`GET /backtests/{run_id}/attribution?by=greek|component|role|underlying`	Attribution tab	Layer 5 PnL and GreeksAttribution
`GET /backtests/{run_id}/events`	Risk events, constraint vetoes, rejects and partial fills, invariant results	Layers 3, 4, 5

Endpoint or field	Purpose	Backend source
`POST /backtests/{run_id}/research` with `type=ablation|swap|counterfactual|baseline|robustness`	Research actions and the robustness set as one call that returns the created run ids	Layer 6 tools
`POST /preview`	Composer preview: short engine pass with decision log	Layer 5 Runner
`POST /backtests/{run_id}/resume`	Resume from a state snapshot	Layer 5 Runner
`GET /ideas`, `GET /ideas/{id}`	Idea grouping, trial count, hold-out status, research stage	ExperimentTracker
`POST /ideas/{id}/reveal-holdout`	Logged one-time reveal	ExperimentTracker
`GET /configs/{id}/versions`, diff	Config history and diff	Config store or git
`GET /datasets/{id}/quality`	Data quality page	Layer 1 quality reports
`GET /surfaces/{underlying}?date=`	Surface view	Layer 1 SurfaceBuilder
Job events (SSE or WebSocket), or polling	Job tray progress	JobQueue
`mode` field on runs	Backtest, paper and live in one analysis page	Layer 7 LiveAdapter

One change to the draft

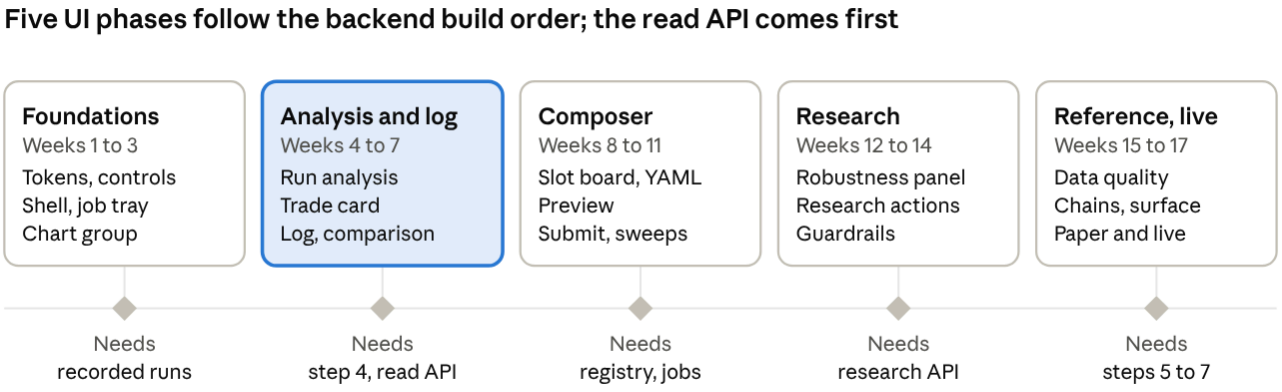
`DELETE /backtests/{run_id}` should cancel a queued or running job and archive a finished run, never remove it. A deleted run would drop out of the idea's trial count and inflate the Deflated Sharpe, which the research protocol depends on.

Conventions

- Every response carries `run_id`, config hash, code commit and data snapshot ID, so the realism strip can render from any endpoint
- Large series are paginated or served as Parquet for charts; the UI requests only the brushed range when zoomed
- All numbers are computed server-side; the UI performs no aggregation beyond display formatting

15. Roadmap

The architecture's build order puts the platform (REST API, job queue, web UI) at step 6. Because the web UI is the primary entry point, this plan pulls the read half of the API forward to sit right after step 4, and starts UI foundations in parallel with steps 3 and 4 using recorded runs.



UI roadmap · 5 phases, each gated on a backend dependency

Phase	Scope	Backend dependency	Gate to pass
1 Foundations	Tokens, base controls, shell, job tray, time-axis chart group, Storybook	Exported results of a few real runs (equity, snapshots, TradeBook) as fixtures	Shell navigable; chart group renders a 5-year run with 1,000 trades smoothly

Phase	Scope	Backend dependency	Gate to pass
2 Analysis and log	Run analysis (all tabs except Robustness), trade card, experiment log, comparison	Step 4 done (TradeBook, tracker, metrics); read endpoints from section 14	3 clicks from a metric card to the component action behind a trade
3 Composer	Slot board, schema forms, YAML view, settings panel, preview, submit, sweep builder	`GET /components`, `POST /backtests`, `POST /sweeps`, job queue, `POST /preview`	2-step change-and-rerun; preview within 10 s
4 Research	Robustness panel, research actions, interaction tests, guardrails, pre-launch checklist	Research endpoint, ideas and trial counts, hold-out reveal	Robustness set in 1 click; trial counts correct after archiving
5 Reference and live	Data quality, chain and surface explorer, component registry page, paper and live modes	Quality and surface endpoints; steps 5 to 7 for paper and live	Walkthroughs in section 16 pass

Week counts assume one designer and two frontend engineers and are estimates. Phases 2 and 3 can overlap once the job queue exists.

How each phase works

1. Build against real outputs from the engine, including extreme cases: long periods, deep drawdowns, thousands of trades, invariant failures, sweeps with dozens of members.
2. Build components and their states in Storybook first, then assemble pages; review the running interface, not mockups.
3. One 30-minute researcher session per week, recording clicks and hesitations; findings go into the next week.
4. Each phase ends with the gate in the table and the matching walkthroughs from section 16.

16. Acceptance criteria and usability testing

Acceptance is judged by measured targets and task walkthroughs. Each phase gate is decided by the targets below that apply to it plus one walkthrough; at least two of three researchers must meet them.

Measured targets

Metric	Target	How measured
Change one slot and submit	⩽ 2 steps	Walkthrough count
Metric card to the component action behind a trade	⩽ 3 clicks	Walkthrough count
Launch the robustness set	1 click	Walkthrough count
Preview response	⩽ 10 s, P95	Telemetry
Analysis page interactive after results are ready	< 2 s for a 5-year run with 1,000 trades	Telemetry, P95
Every number on screen traceable to an endpoint and `run_id`	100%	Code review and automated test
Trial count unchanged by archiving	100%	Automated test
State preserved across tab switches; URL restores view	100%	Automated tests
Core tasks 1 to 4 by keyboard	Without a mouse	Walkthrough
Text contrast	⩾ 4.5 : 1	Automated check
New user composes and runs a config	< 10 min, no training	New-member walkthrough

Walkthrough script

Give the participant a concrete question without hinting at the path; record clicks, time and every pause over 5 s.

1. In the composer, swap the Stop Loss variant for another registered one, run it, and state how the max drawdown and its interval changed versus the baseline.
2. In a given run, find the largest losing trade in a given month, and state which slot closed it, the reason, and the data values that triggered it.
3. Ablate the Headwind slot on a run and state whether the difference is significant.

4. Launch the robustness set on a run and state which item fails.
5. Compare three runs and identify which one differs in data snapshot rather than config.
6. Run counterfactual exits for the Take Profit slot and state the total money given up, and why the result is labeled in-sample.
7. Share the trade from task 2; the recipient opens the same view.
8. New member: reproduce a run and state its commit, data snapshot and result hash.

Per-page checklist

- [ ] One clear focal point; structure recognizable at quarter size
- [ ] All states from section 11 implemented and in Storybook
- [ ] No client-side metric computation; every number tied to an endpoint
- [ ] Realism strip present on every result view
- [ ] Hold-out shaded and blank unless revealed
- [ ] Intervals shown wherever a metric is compared
- [ ] Slot, Greek and leg colors from tokens; no hard-coded values
- [ ] Keyboard reachable with a sensible focus order
- [ ] State consistent after tab switch, refresh, back and forward
- [ ] Contrast checked in dark and light mode

Review cadence

One 45-minute review per week on the running interface; problems are recorded, not solved in the room, and ranked as blocks a task, adds steps, inconsistent, cosmetic. The first two tiers enter the next week.

17. Open decisions

Decided: the web UI is the primary, interactive entry point for researchers. The decisions below have proposed defaults that this plan already assumes; each can be confirmed or changed without restructuring the plan.

Decision	Proposed default	Why it matters
Realism assumptions in the UI	Read-only; varied only through robustness runs; bid or mid mark is the only display toggle	Keeps the architecture's rule that realism assumptions are tested, never tuned

Decision	Proposed default	Why it matters
Preview latency and endpoint	`POST /preview` with a 60-day warm window, target ⩽ 10 s	Components are stateful; a cheaper preview would show wrong behavior
Strictness of protocol guardrails	Hard: hold-out locked per idea and revealed once with a log entry; archived runs count as trials	Low-frequency option strategies have few trades and fat tails, so overfitting is the main risk
Pass thresholds for robustness items	Team-level settings, edited by a lead, not per run	A pass must mean the same thing for every run
`DELETE` behavior	Cancel or archive only	Deleting runs would corrupt the trial count and the Deflated Sharpe
Read API timing	Pulled forward to right after backend step 4	The analysis page is the most valuable surface and needs only reads
Job progress transport	Server-sent events, polling as fallback	Smooth progress in the job tray without load
Paper and live modes	Designed now (`mode` on runs), built in phase 5	Avoids restructuring the analysis page later
Single user or team	Team: shared links, roles, audit log via the Auth layer	Links and reviews are central to the workflow
Variant authoring in the browser	Out of scope; variants are code with unit tests, written outside the UI	Keeps component quality under test and review

The architecture's own open decisions also shape the UI: option quote granularity and snapshot times decide what the realism strip shows; `check_frequency` and hedge instruments decide which settings the composer exposes; the underlying universe decides whether earnings markers and per-underlying checks are prominent; hold-out length and minimum trade count feed the guardrails directly.

18. Appendix

Shortcut table

Cmd on Mac, Ctrl on Windows; shortcuts appear in button tooltips.

Scope	Shortcut	Action
Global	Cmd + K	Command palette
Global	Cmd + Enter	Submit the current config
Global	Cmd + Shift + C	Copy a link to the current view
Global	[and]	Previous and next run in the idea
Global	1 to 6	Switch analysis tab
Global	Esc	Close drawer or popover, clear brush
Table	Up and down arrows	Move the current row
Table	Enter	Open the trade card
Table	Space	Tick the current row
Table	Shift + arrows	Extend selection
Trade card	J and K	Next and previous trade
Trade card	F	Toggle full screen
Chart	Drag	Brush a time range
Chart	Double-click	Reset the time range
Chart	L	Toggle log scale
Chart	M	Toggle bid or mid mark (display only)
Composer	Cmd + S	Save draft config
Composer	Cmd + P	Run preview
Composer	Cmd + /	Switch between slot board and YAML
Composer	Alt + 1 to 7	Open slot card 1 to 7

Design token table

Token	Value	Use
space-1 to space-5	4, 8, 12, 16, 24 px	Spacing within and between components
space-6, space-7	32, 48 px	Between blocks and page sections

Token	Value	Use
font-xs, sm, base, md, lg, xl	11, 12, 13, 14, 16, 20 px	Type scale
weight-regular, medium	400, 500	The only two weights
radius-sm, md, lg	6, 8, 12 px	Controls, cards, drawers
row-compact, row-comfortable	28, 36 px	Table row heights
surface-0, 1, 2	Dark: #111, #181818, #1f1f1f; light: #f7f7f5, #ffffff, #ffffff	Page, panel, card
text-primary, secondary, muted	Dark: #e8e8e6, #a0a09c, #6b6b68; light: #1f1f1e, #5f5e5a, #8a8985	Text levels
border, border-strong	Dark: rgba(255,255,255,0.08), 0.16; light: rgba(0,0,0,0.08), 0.16	Dividers
accent	#378add	Primary actions, selection, links
success, warning, danger	#3b9f5a, #d9931e, #e24b4a	Status, pass and fail, P&L sign
data-strategy, data-benchmark	#378add, #888780	Equity line, benchmark
slot-1 to slot-7	#7f77dd, #1d9e75, #d85a30, #d4537e, #ba7517, #185fa5, #5f8f3a	Setup, Entry, Take Profit, Stop Loss, Headwind, Positioning, Hedge
greek-delta, gamma, vega, theta, residual	#378add, #7f77dd, #1d9e75, #d85a30, #888780	Exposure and attribution
leg-main, leg-hedge	#378add, #ba7517	Leg roles
data-1 to data-4	#378add, #d85a30, #1d9e75, #7f77dd	Comparison columns
holdout-fill	45° hatch at 12% of text-muted	Locked hold-out period
motion-fast, base	150, 200 ms, ease-out	Hover, drawer and panel transitions
drawer-width, drawer-max	400, 560 px	Drawer widths

Token	Value	Use
nav-collapsed, nav-expanded	56, 240 px	Left rail states
min-content-width	1100 px	Center area minimum

Color values are a starting proposal; verify each against the contrast, saturation and color-blind rules in section 12 before finalizing.